Military Institute of Science and Technology

Department of Computer Science and Engineering



A Learned Selector for Classical Disk Scheduling

Term Paper – Track 2

Student Name: Aria Nawshin Ashka

Student ID: 202414021

Course Code: CSE-307

Course Title: Operating Systems

October 2026

Abstract

No fixed disk-scheduling policy is best for every request pattern. This paper implements FCFS, SSTF, SCAN, and C-SCAN and trains a small decision tree to select a scheduler for each 50-request window, together with a confidence score. The simulator uses tracks 0–199, a head reset to 50, and course-convention SCAN/C-SCAN; sequential, random, and bursty workloads are generated with fixed seeds, with disjoint training, validation, and test seeds. On 660 held-out windows the tree reaches 75.61% tie-aware accuracy and 124,574 tracks of total seek, which is 6.53% below the best fixed policy (SSTF, 133,276) and close to the offline oracle (119,296). Within any single workload, however, it only nearly matches the best fixed scheduler, and it is weakest on sequential workloads. Confidence is higher on correct (0.877) than on wrong (0.778) predictions, but the expected calibration error of 0.148 shows overconfidence. Under sequential→random and random→bursty shifts, regret does not rise after the change; it tracks how hard each workload type is. A validation-selected random-forest extension improves calibration but not seek regret. The study uses seek distance only and synthetic traces.

Keywords: disk scheduling, FCFS, SSTF, SCAN, C-SCAN, decision tree, calibration

1 Introduction and System Design

Disk scheduling determines the order in which a device services pending track requests. FCFS preserves arrival order but can travel unnecessarily; SSTF greedily chooses the nearest request and may starve distant requests; SCAN sweeps like an elevator; and C-SCAN provides a one-directional sweep with a wrap [1]. Their relative cost depends on locality and request order, motivating an adaptive selector.

The simulator uses tracks 0–199. In the main experiment the head resets to 50 for every independent, non-overlapping 50-request window. Following the course convention, SCAN initially moves toward 0, reaches physical track 0, then reverses. C-SCAN also moves toward 0, reaches 0, wraps to 199, and counts that wrap. Unit tests reproduce the course queue $[98, 183, 37, 122, 14, 124, 65, 67]$ from head 53: FCFS=640, SSTF=236, SCAN=236, and C-SCAN=386.

For each window, all four algorithms run offline. The minimum-cost algorithm is the oracle label. All tied winners are stored; exact ties are excluded only from primary classifier fitting. The main classifier is an interpretable decision tree (depth 5, minimum leaf size 12, balanced class weights, fixed seed), a standard classification method [2]. It receives eight pre-decision features: variance, range, mean absolute consecutive gap, gap coefficient of variation, direction consistency, mean distance from the head, and fractions below and above the head. Confidence is the maximum predicted class probability.

2 Experimental Method

Sequential workloads are mostly monotonic with small positional noise; random workloads are uniform over all tracks; and bursty workloads contain ordered clusters around several hot spots. Twenty windows per workload are generated for every seed. Seeds 1–20 train the model, 21–30 form a validation set, and 100–110 are held out for final testing. This yields 1,200 training and 660 test windows. The training set contains 63 unique FCFS, 267 SCAN, and 800 SSTF labels, plus 70 ties. C-SCAN never uniquely wins.

Two 40-window stress timelines change after window 20: sequential→random and random→bursty. Metrics are total seek, tie-aware accuracy, selector regret (selected cost minus oracle cost), confusion matrix, confidence on correct versus wrong cases, a reliability diagram, and 10-bin ECE [3]. The implementation uses Python, NumPy, pandas, Matplotlib, and scikit-learn [4]; raw CSVs and fixed seeds make every result reproducible.

Table 1: Total seek on all 660 held-out windows. "Always SSTF" is the strongest single fixed-policy baseline.

Policy	Total seek	Policy	Total seek
FCFS	902,104	C-SCAN	251,307
Always SSTF	133,276	Oracle	119,296
SCAN	144,737	Learned selector	**124,574**

3 Results and Analysis

Table 1 shows that, across all windows, the selector saves 8,702 tracks (6.53%) relative to always choosing SSTF. It is only 5,278 tracks above the oracle over all windows, or 8.00 tracks per window. The 597 non-tied windows give 74.37% ordinary accuracy; accepting any tied oracle winner gives 75.61% over all 660 windows. Classification accuracy alone is insufficient: a wrong choice may have almost the same seek cost as the winner, while a less frequent error can be expensive.

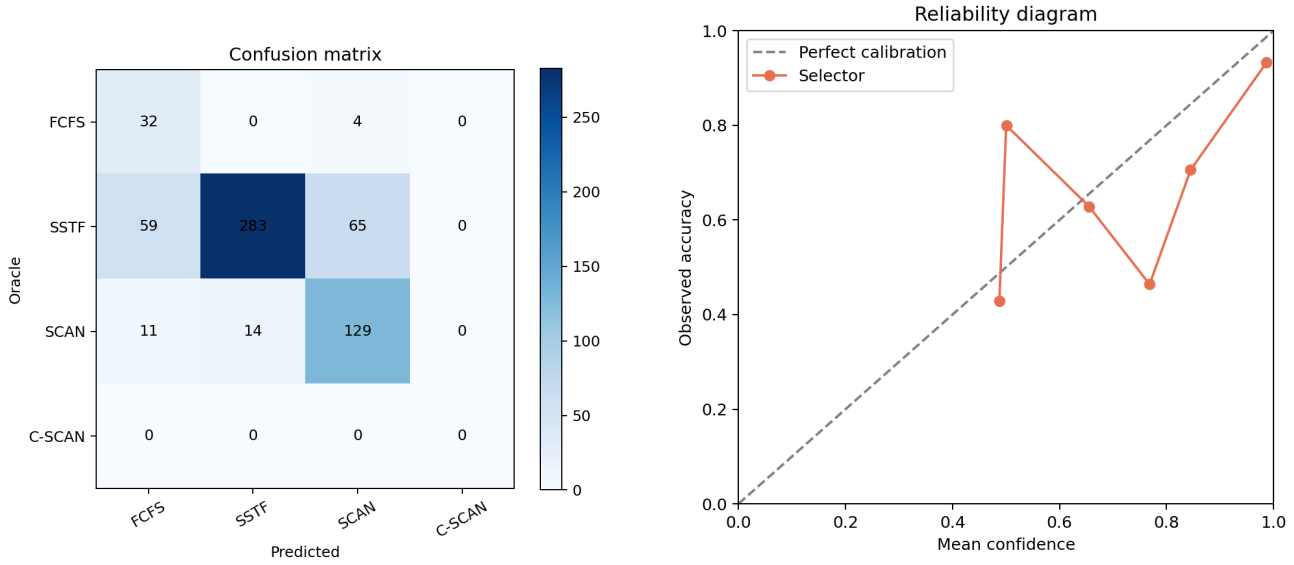
Table 2: Per-workload held-out seek totals for 220 windows each.

Workload	FCFS	SSTF	SCAN	C-SCAN	Oracle	Selector	Acc.
Sequential	44,549	38,504	46,291	84,450	35,020	39,214	62.7%
Random	729,441	63,558	54,054	86,664	53,398	54,098	70.9%
Bursty	128,114	31,214	44,392	80,193	30,878	31,262	93.2%

Table 2 exposes the main weakness rather than hiding it. The overall gain over SSTF exists because the best fixed policy differs by workload (SCAN on random, SSTF on sequential and bursty); within each single workload the selector does not beat the best fixed policy, but comes within 0.1% (random), 0.2% (bursty), and 1.8% (sequential) of it. Sequential accuracy is only 62.7%, with mean regret 19.06, compared with 3.18 for random and 1.75 for bursty workloads. The confusion matrix in Fig. 1 shows most errors involve SSTF versus FCFS or SCAN.

C-SCAN never wins under this experimental convention because the head begins near the lower end (50), movement is initially toward 0, and the complete 0→199 wrap is charged. That fixed wrap is expensive for these finite windows. This is a result of the stated geometry and cost model, not a claim that C-SCAN is universally inferior; fairness and starvation are not included in the objective.

Confidence is meaningfully informative: the main tree averages 0.877 when correct and 0.778 when wrong. However, ECE=0.148 and the reliability curve show imperfect calibration.



(a) Non-tied confusion matrix.

(b) Reliability diagram.

Figure 1: Diagnostic evidence for classification and confidence.

3.1 Behavior under workload shift

Fig. 2 plots mean regret and confidence per window across the 11 test seeds, with the dashed line at the shift. Under sequential→random, mean regret falls from 19.06 to 3.18 after the change while confidence falls from 0.861 to 0.809. Under random→bursty, regret falls from 4.32 to 1.49 and confidence rises from 0.799 to 0.889. The shift therefore does *not* degrade the selector. The improvement after the first shift occurs because the sequential phase is the hardest workload for the model (62.7% accuracy), not because the shift helps; the second shift moves into the easiest workload (bursty, 93.2%). Because the selector classifies each window independently, there is no adaptation lag at the boundary, and regret changes only through workload difficulty. These are independently generated windows concatenated at the shift, not one continuous trace. Confidence stays imperfectly calibrated, so it should not be interpreted as a guarantee.

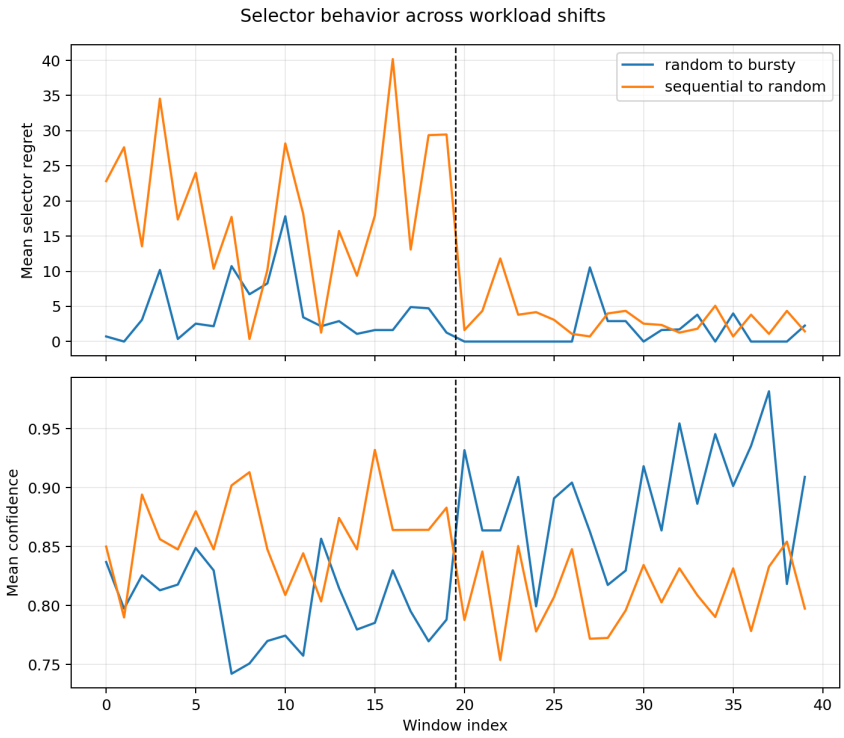


Figure 2: Mean selector regret and confidence per window across the two shift timelines (mean of 11 test seeds); dashed line marks the workload shift after window 20.

4 Validation-Controlled Extensions

Four additional features target sequential structure: absolute correlation between request index and track, fraction of consecutive gaps at most five, median gap, and maximum gap. A logistic-regression baseline and a 300-tree random forest were trained on seeds 1–20. Model choice used only validation seeds 21–30; the random forest won with 78.5% validation accuracy and ECE 0.028. It was then evaluated once on the held-out test set.

The forest reaches 79.09% tie-aware test accuracy, 70.91% sequential accuracy, and ECE 0.051; confidence separates correct and wrong predictions more clearly (0.827 versus 0.615). Yet its mean regret is 8.90 and total seek is 125,170, slightly worse than the main tree. Logistic regression is weaker at 73.33% accuracy and 10.44 mean regret. Therefore the tree remains the primary result: it is simpler to explain and minimizes the measured seek objective, while the forest is a useful calibration-focused extension.

A window-size sensitivity check retrains the tree at 30, 50, and 100 requests. Tie-aware accuracy is 81.52%, 75.61%, and 84.39%, respectively; mean regret is 4.05, 8.00, and 2.95. SSTF remains the best fixed scheduler at each size, and the selector remains better than that fixed baseline across all windows. The exact performance is sensitive to window length, but the principal ranking is unchanged.

5 Limitations and Conclusion

This simulation measures seek distance only. It omits rotational latency, caching, SSD behavior, concurrent arrivals during service, priorities, starvation, and device-level timing. Complete windows and an offline oracle simplify a real online problem. The fixed head and fixed SCAN direction also limit generalization. Future work should use rolling windows, multiple head positions, cost-sensitive training, and calibration learned on a dedicated calibration set.

Within these limits, the learned tree beats every fixed scheduler in total seek across all windows, because the best fixed policy changes with the workload, and it comes close to the oracle while exposing useful confidence information. Within a single workload type it only nearly matches the best fixed policy. The results also show an important systems lesson: the most accurate classifier need not minimize operational cost. All code, seeds, raw outputs, figures, tests, and an interactive custom-queue demonstration are included for audit and reproduction.

AI-Use Disclosure

Generative AI assisted with scaffolding, implementation, documentation, and debugging. All numerical claims in this paper were regenerated by deterministic code and checked against committed CSV/JSON outputs. The student remains responsible for understanding and defending the work.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.

[2] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[3] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proc. ICML*, 2017, pp. 1321–1330.

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.